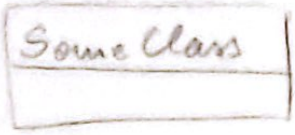
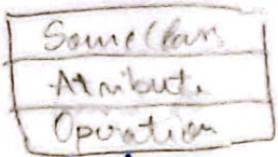
TEST 4 FIS

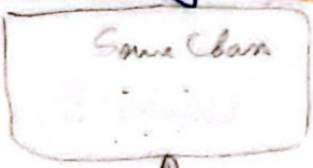
UML class Diagrams and Sequence Diagrams

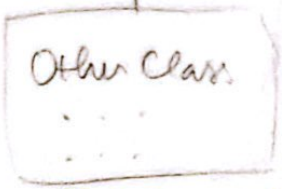


Class



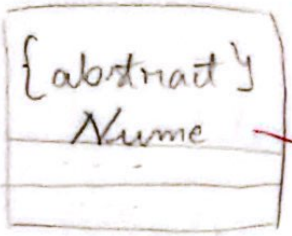
Features





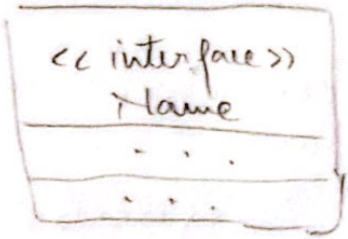
Relationships

```
class Some Class {
} ...
```




```
abstract class Name {
} ...
```

italic



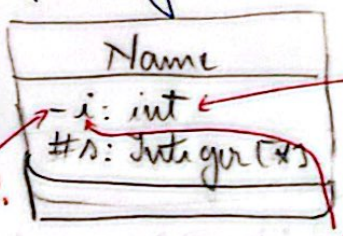

```
interface Name {
} ...
```

Atributes

\+ public
\- private
\# protected
~ package

1 exactly one
0...1 zero or at most one
0...* zero or more but NO upper limit
sau *

Ex:



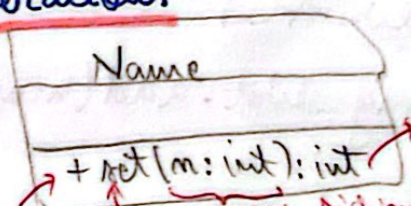
tipuri

vizibil.

Name var.

```
class Name {
    private int i;
    protected List<Integer> s;
}
```

Operations



tip returnat.

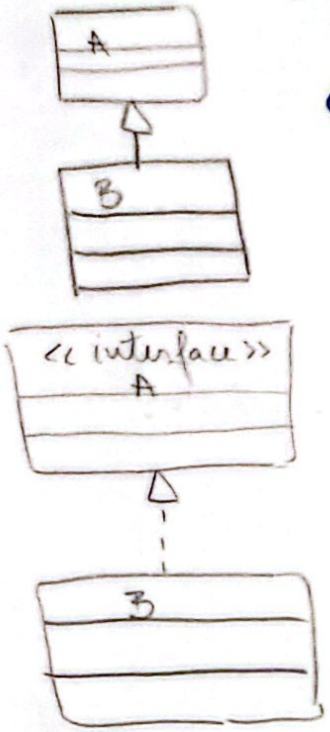
vizib. nume metoda param. si tipul lor

```
class Name {
    public int set(int n){
    } ...
}
```

Scanned with CamScanner

! dacă o var. / metodă este subliniată este statică

Relationship



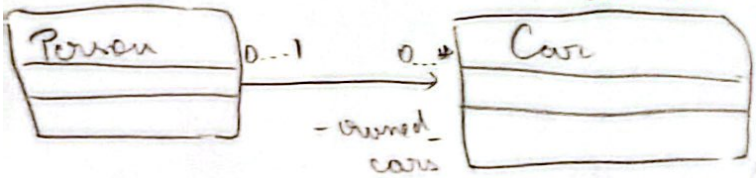
Class B extends A {...}

Class B implements A {...}

Asociere

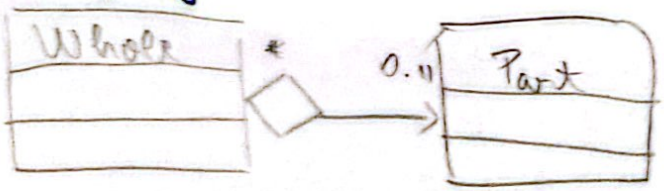
→ la asociere, agregare, compoziție sunt obiecte create în clase

→ la dependință doar se apelează / se dau ca param. la fct.




```
class Person {
private List<Car> owned
 cars = new ArrayList<>()
}
```

Agregare



Similar

Compoziție



No-sharing

Obiectul B nu există fără A.

```
class A {
 private List<B> my_list=...
 public A()
 {
  my_list.add(new B());
 }
}
```

dependente

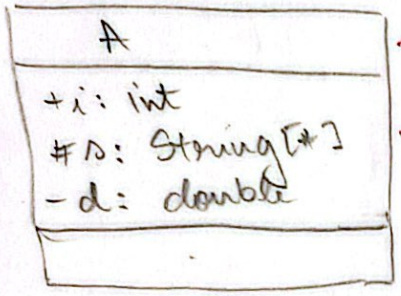



```
class A {
  public void m (Bx){
      x.doS();
  }
}
```

GRILE

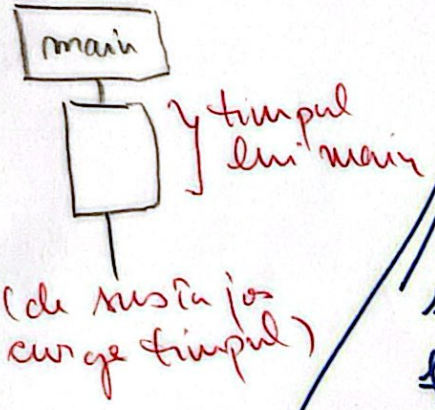
1) • La interfață se pune <<...>>
   • Liniile pt. extends și implements sunt diferite.
   • Metodele statice sunt subliniate.
   • Metodele abstracte sunt scrise înclinat.

2).



→ colecția s e protected
→ clasa A are 3 atribute:
public int i
privată double d
o listă s fără limită

3). Sequence Diagram → arată cum funcționează
→ ne arată structura
→ capturează interacțiunea dintre obiecte
→ ne arată relațiile dintre clase (diagrama de clasă face asta).
→ sunt concentrate în timp, îl scriem folosind axa verticală
→ detaliază modul în care sunt rel. operațiunile



4) ✓

Scanned with CamScanner